Java Programming Fundamentals

Comprehensive Lecture Notes, Code Examples & Practice Assessment

Target Audience: College Computer Science / Engineering Students **Module:** Core Syntax & Operators

1. Floating-Point Literals & Scientific Notation

In Java, real numbers containing fractional parts are represented using floating-point types: `float` (single precision, 32-bit) and `double` (double precision, 64-bit). By default, any floating-point literal typed into a Java program (e.g., `5.0`) is treated as a `double` value.

Explicit Suffixes

To specify the exact data type of a floating-point literal, Java provides explicit type suffixes:

- **Float Literal:** Append an `f` or `F` (e.g., `100.2f` or `100.2F`).
- **Double Literal:** Append a `d` or `D` (e.g., `100.2d` or `100.2D`). Alternatively, omitting the suffix defaults to `double`.

> **Precision Comparison:** `double` provides 64-bit precision (approx. 15-17 significant decimal digits), making it significantly more accurate than `float`, which provides 32-bit precision (approx. 6-7 significant decimal digits).
>
> *Example Output Comparison:*
> `System.out.println("1.0 / 3.0 is " + 1.0 / 3.0);` → `1.0 / 3.0 is 0.3333333333333333`
> `System.out.println("1.0F / 3.0F is " + 1.0F / 3.0F);` → `1.0F / 3.0F is 0.33333334`

Scientific Notation Syntax

Floating-point literals can be expressed using scientific notation in the standard mathematical form $a \times 10^b$. In Java, the symbol `E` (or `e`) replaces the "$\times 10^b$" exponent multiplier:

- `1.23456 × 10^2` is written as `1.23456E2` or `1.23456E+2` (Value: `123.456`)
- `1.23456 × 10^-2` is written as `1.23456E-2` (Value: `0.0123456`)

> **Why are they called "Floating-Point" numbers?**
> Internally, these numbers are stored in scientific notation (binary IEEE 754 standard). When a number like `50.534` is normalized into scientific notation (`5.0534E+1`), its decimal point is "floated" to a position immediately after the first non-zero digit.

Topic 1 Self-Assessment: Floating-Point Literals

EASY **1. What is the default data type of the literal `12.34` in Java?**

A) float
B) double
C) BigDecimal
D) int

EASY **2. Which suffix is used to explicitly declare a literal as a single-precision 32-bit float?**

A) d or D
B) f or F
C) s or S
D) l or L

MEDIUM **3. Which of the following expressions represents the scientific notation equivalent of `0.052534`?**

A) 5.2534e+1
B) 5.2534e-2
C) 0.52534e+2
D) 525.34e-1

MEDIUM **4. Which of the following is NOT a valid floating-point literal in Java?**

A) 12.3e+2
B) -334.4
C) 39F
D) 40D_e2

HARD **5. What is printed by `System.out.println(1.0f / 3.0f == 1.0 / 3.0);`?**

A) true
B) false
C) Compilation Error
D) Runtime Exception

Answers & Explanations:
1. **B** (Unsuffixed floating-point literals are always `double`). | 2. **B** (Letter `f` or `F`). | 3. **B** (`5.2534 × 10^-2 = 0.052534`). | 4. **D** (Malformed scientific notation). | 5. **B** (`float` and `double` have different precision levels, resulting in unequal binary representations).

2. Evaluating Expressions and Operator Precedence

Java evaluates arithmetic expressions using standardized mathematical precedence rules. Complex mathematical expressions are translated directly into Java statements using corresponding operators (`+`, `-`, `*`, `/`, `%`).

Translating Mathematical Formulas to Java Syntax

Consider the algebraic expression:

$$f(x, y, a, b, c) = ¾ \times \frac{3 + 4x}{5} - \frac{10(y - 5)(a + b + c)}{x} + 9 \times \left(\frac{4}{x} + \frac{9 + x}{y} \right)$$

In Java syntax, explicit multiplication operators (`*`) and grouping parentheses (`()`) must be supplied:

```
(3 + 4 * x) / 5 - 10 * (y - 5) * (a + b + c) / x + 9 * (4 / x + (9 + x) / y)
```

Operator Precedence & Evaluation Order

- **Parentheses `()`:** Evaluated first from innermost to outermost pairs.
- **Multiplicative Operators (`*`, `/`, `%`):** Higher precedence; evaluated left to right.
- **Additive Operators (`+`, `-`):** Lower precedence; evaluated left to right.

Step-by-Step Evaluation Breakdown

Expression: `3 + 4 * 4 + 5 * (4 + 3) - 1`

Step	Operation Performed	Resulting Expression
1	Evaluate Parentheses `(4 + 3)`	`3 + 4 * 4 + 5 * 7 - 1`
2	First Multiplication `4 * 4`	`3 + 16 + 5 * 7 - 1`
3	Second Multiplication `5 * 7`	`3 + 16 + 35 - 1`
4	First Addition `3 + 16`	`19 + 35 - 1`
5	Second Addition `19 + 35`	`54 - 1`
6	Final Subtraction `54 - 1`	**`53`**

Topic 2 Self-Assessment: Operator Precedence

EASY **1. What is the value of the expression `46 / 9` in Java integer arithmetic?**

A) 5.1111
B) 5
C) 5.11
D) 1

EASY **2. What is the result of `46 % 9` ?**

A) 5
B) 1
C) 0
D) 9

MEDIUM **3. Evaluate the expression: `a = 46 % 9 + 4 * 4 - 2;`**

A) 15
B) 18
C) 20
D) 13

MEDIUM **4. Evaluate the expression: `a = 45 + 43 % 5 * (23 * 3 % 2);`**

A) 48
B) 45
C) 51
D) 46

HARD **5. Which of the following statements regarding Java expressions and assignment is TRUE?**

A) Any expression can be used standalone as a statement.
B) The assignment `x = y = x = 0;` is illegal in Java.
C) The expression `x++` can be legally used as a statement.
D) The statement `x = x + 5` has no return value.

Answers & Explanations:
1. **B** (Integer division truncates decimals: 46 / 9 = 5). | 2. **B** (Remainder operator: 46 divided by 9 is 5 with remainder 1). | 3. **A** (46 % 9 = 1; 4 * 4 = 16; 1 + 16 - 2 = 15). | 4. **A** (Inside parens: (69 % 2) = 1; 43 % 5 = 3; 3 * 1 = 3; 45 + 3 = 48). | 5. **C** (Increment statements like `x++` are valid expression statements; multiple assignment `x = y = x = 0` is legal).

3. Case Study: Time Computation & System Utilities

Java provides the method `System.currentTimeMillis()` which returns the current elapsed time in **milliseconds** since midnight, **January 1, 1970 GMT**. This reference point is known as the *UNIX Epoch*.

Step-by-Step Algorithm to Extract Current GMT Time

1. **Total Milliseconds:** `long totalMilliseconds = System.currentTimeMillis();`
2. **Total Seconds:** Divide total milliseconds by 1,000 → `long totalSeconds = totalMilliseconds / 1000;`
3. **Current Second:** Take remainder modulo 60 → `long currentSecond = totalSeconds % 60;`
4. **Total Minutes:** Divide total seconds by 60 → `long totalMinutes = totalSeconds / 60;`
5. **Current Minute:** Take remainder modulo 60 → `long currentMinute = totalMinutes % 60;`
6. **Total Hours:** Divide total minutes by 60 → `long totalHours = totalMinutes / 60;`
7. **Current Hour (GMT):** Take remainder modulo 24 → `long currentHour = totalHours % 24;`

Complete Executable Program

```
public class ShowCurrentTime {
    public static void main(String[] args) {
        // Obtain the total milliseconds since midnight, Jan 1, 1970
        long totalMilliseconds = System.currentTimeMillis();

        // Obtain total seconds
        long totalSeconds = totalMilliseconds / 1000;

        // Compute current second in the minute
        long currentSecond = totalSeconds % 60;

        // Obtain total minutes
        long totalMinutes = totalSeconds / 60;

        // Compute current minute in the hour
        long currentMinute = totalMinutes % 60;

        // Obtain total hours
        long totalHours = totalMinutes / 60;

        // Compute current hour (24-hour clock)
        long currentHour = totalHours % 24;

        // Display results
        System.out.println("Current time is " + currentHour + ":"
            + currentMinute + ":" + currentSecond + " GMT");
    }
}
```

Topic 3 Self-Assessment: System Time & Modular Math

EASY **1. What timestamp reference point does the UNIX Epoch represent?**

A) Midnight, Jan 1, 2000 GMT
B) Midnight, Jan 1, 1970 GMT
C) Midnight, Jan 1, 1980 GMT
D) System boot time

EASY **2. Which return data type is produced by `System.currentTimeMillis()`?**

A) int
B) double
C) long
D) float

MEDIUM **3. If total seconds is `3665`, what will be computed as `currentSecond` using `totalSeconds % 60`?**

A) 5
B) 60
C) 1
D) 65

MEDIUM **4. How do you convert the calculated 24-hour `currentHour` in GMT to EST (UTC-5)?**

A) (currentHour + 5) % 24
B) (currentHour - 5 + 24) % 24
C) currentHour * 5
D) currentHour / 5

HARD **5. What happens if you store `System.currentTimeMillis()` into an `int` variable?**

A) Implicit widening occurs automatically
B) Compiler error due to loss of precision
C) Runtime exception
D) Value is rounded automatically

Answers & Explanations:
1. **B** (Jan 1, 1970 GMT). | 2. **C** (Returns 64-bit integer `long`). | 3. **A** (3665 / 60 = 61 mins with remainder 5 seconds). | 4. **B** (Adding 24 prevents negative numbers before modulo arithmetic). | 5. **B** (Assigning a 64-bit `long` to a 32-bit `int` without an explicit cast causes a compile error).

4. Augmented Assignment Operators

Augmented (or compound) assignment operators combine binary arithmetic operations with assignment into a single concise expression.

Operator	Name	Example Code	Equivalent Standard Statement
`+=`	Addition Assignment	`i += 8`	`i = i + 8`
`-=`	Subtraction Assignment	`i -= 8`	`i = i - 8`
`*=`	Multiplication Assignment	`i *= 8`	`i = i * 8`
`/=`	Division Assignment	`i /= 8`	`i = i / 8`
`%=`	Remainder Assignment	`i %= 8`	`i = i % 8`

Implicit Type Casting Rule in Compound Assignments:
In Java, an augmented expression of the form `x1 op= x2` is implicitly implemented as:

`x1 = (T)(x1 op x2)`

where `T` is the data type of `x1`.

Example:
```
int sum = 0;
sum += 4.5; // Compiles legally! Equivalent to: sum = (int)(sum + 4.5);
```
The result of `sum` becomes `4` (truncated), automatically performing the narrow explicit cast without throwing a compiler error.

Topic 4 Self-Assessment: Augmented Assignment

EASY **1. What is the value of `x` after `int x = 10; x %= 3;`?**

A) 3
B) 1
C) 0
D) 3.33

EASY **2. Statement `x *= a + 1` is equivalent to which full expression?**

A) x = x * a + 1
B) x = x * (a + 1)
C) x = a + 1
D) x = x + a * 1

MEDIUM **3. Given `int a = 1;` and `a %= 3 / a + 3;`, what is the value of `a`?**

A) 1
B) 0
C) 3
D) 4

MEDIUM **4. What will be stored in `int x = 5; x /= 2;`?**

A) 2.5
B) 2
C) 3
D) 0

HARD **5. What happens when compiling: `int x = 10; double y = 2.5; x *= y;`?**

A) Compile Error: Loss of precision
B) Compiles clean; x becomes 25
C) Runtime Exception
D) Compiles clean; x becomes 25.0

Answers & Explanations:
1. **B** (10 % 3 = 1). | 2. **B** (Right hand side is evaluated completely with implicit parens first). | 3. **A** (Right side: 3/1 + 3 = 6. 1 %= 6 yields 1). | 4. **B** (Integer division truncates to 2). | 5. **B** (Implicit casting converts it to `x = (int)(x * y)` → 25).

5. Increment and Decrement Operators

The increment (`++`) and decrement (`--`) shorthand operators increase or decrease a variable's value by `1`. They exist in two formats: **Prefix** and **Postfix**.

Operator	Name	Description	Example Code Execution (Assume `i = 1`)
`++var`	Pre-increment	Increments `var` by 1 first, then uses the new value in statement.	`int j = ++i;` → `j` is **2**, `i` is **2**
`var++`	Post-increment	Uses current value in statement first, then increments `var` by 1.	`int j = i++;` → `j` is **1**, `i` is **2**
`--var`	Pre-decrement	Decrements `var` by 1 first, then uses the new value in statement.	`int j = --i;` → `j` is **0**, `i` is **0**
`var--`	Post-decrement	Uses current value in statement first, then decrements `var` by 1.	`int j = i--;` → `j` is **1**, `i` is **0**

Topic 5 Self-Assessment: Increment & Decrement

EASY **1. What are the values of `x` and `y` after: `int x = 5; int y = ++x;` ?**

A) x = 5, y = 6
B) x = 6, y = 6
C) x = 6, y = 5
D) x = 5, y = 5

EASY **2. What are the values of `x` and `y` after: `int x = 5; int y = x--;` ?**

A) x = 4, y = 5
B) x = 4, y = 4
C) x = 5, y = 4
D) x = 5, y = 5

MEDIUM **3. Given `double d = 1.0; int a = 1;`, what is the output of `d += 1.5 * 3 + (++a);` ?**

A) 6.5
B) 7.5
C) 5.5
D) 8.0

MEDIUM **4. Given `double d = 1.0; int a = 1;`, what is the output of `d -= 1.5 * 3 + a++;` ?**

A) -4.5
B) -5.5
C) -3.5
D) 0.0

HARD **5. What is the value of `k` after executing: `int i = 2; int k = ++i + i++ * --i;` ?**

A) 11
B) 12
C) 9
D) 10

Answers & Explanations:
1. **B** (Pre-increment increases `x` to 6 first, then assigns 6 to `y`). | 2. **A** (Post-decrement assigns original 5 to `y` first, then decrements `x` to 4). | 3. **B** (`++a` makes a=2; 1.5*3=4.5; 4.5+2=6.5; d = 1.0 + 6.5 = 7.5). | 4. **A** (`a++` uses original a=1; 1.5*3=4.5; 4.5+1=5.5; d = 1.0 - 5.5 = -4.5). | 5. **B** (Step 1: `++i` → i=3, term=3. Step 2: `i++` → term=3, i becomes 4. Step 3: `--i` → i=3, term=3. Computation: 3 + (3 * 3) = 12).

6. Numeric Type Conversions and Casting

Binary operations require operands to be of compatible types. If operands differ in data type, Java performs automatic conversion (widening) or requires explicit type casting (narrowing).

Automatic Type Widening vs. Explicit Narrowing

- **Widening Conversion:** Converting a small range type to a larger range type (e.g., `int` to `double`, or `long` to `float`). Handled automatically by Java without data loss.
- **Narrowing Conversion:** Converting a large range type to a smaller range type (e.g., `double` to `int`). Requires explicit casting syntax: `(targetType) value`. Truncates fractional data.

```
// Example 1: Truncation during narrowing cast
System.out.println((int)1.7); // Displays 1 (fractional .7 is lost)

// Example 2: Explicit cast altering division mode
System.out.println((double)1 / 2); // Displays 0.5 (1 cast to 1.0, then 1.0 / 2.0 = 0.5)
System.out.println(1 / 2);         // Displays 0 (Integer division truncates fraction)
```

Immutability of Cast Variable:
Casting creates a temporary value of the new type; it does **NOT** change the original variable's stored value or data type.

```
double d = 4.5;
int i = (int)d; // i becomes 4, but d remains 4.5
```

Topic 6 Self-Assessment: Numeric Type Conversions

EASY **1. What is the output of `System.out.println((int) 3.9);`?**

A) 4
B) 3
C) 3.9
D) Compile Error

EASY **2. What type of conversion takes place when assigning an `int` value to a `double` variable?**

A) Narrowing Casting
B) Automatic Widening Conversion
C) Parse Error
D) Explicit Casting

MEDIUM **3. What is printed by `System.out.println((double)(1 / 2));`?**

A) 0.5
B) 0.0
C) 0
D) 0.50

MEDIUM **4. Which expression correctly performs floating-point division on two integer variables `a` and `b`?**

A) (double) (a / b)
B) (double) a / b
C) a / (int) b
D) (int) a / (int) b

HARD **5. What happens when assigning a `long` value to a `float` variable without explicit casting?**

A) Compile Error: Incompatible types
B) Automatically widens because float range is larger
C) Runtime Overflow Exception
D) Data truncates automatically

Answers & Explanations:
1. **B** (Trimming truncates fractional part, leaving 3). | 2. **B** (Widening happens implicitly because double holds a larger range). | 3. **B** (Integer division 1/2 evaluates to 0 first, then 0 cast to double becomes 0.0). | 4. **B** (Casting `a` to double first ensures floating-point division is used). | 5. **B** (Even though `long` uses 64 bits and `float` uses 32 bits, `float` has a larger value range due to exponent representation).